Universidade de Aveiro

Departamento de Eletrónica, Telecomunicações e Informática

API UA

Technical Report

PECI

Equipa

André Reis: 119814

Vasco Oliveira: 119113

Pedro Ferreira: 119240

Carlos Ramos: 119873

Dinis Malhado: 120420

Simão Pereira: 120459

Orientador: Diogo Gomes

June 4, 2026

Abstract(mudar)

For over a decade, the api.web.ua.pt platform served as a vital digital backbone for the University of Aveiro, centralizing essential services ranging from canteen menus to academic ticketing. However, as the digital landscape evolved, the original infrastructure began to show its age, struggling with outdated security protocols, poor readability, and a lack of modern architectural standards. These mounting technical limitations eventually led to the decommissioning of the legacy system, sparking the urgent need for a complete, from-the-ground-up rebuild to better serve the modern student body. This project outlines the comprehensive modernization of the university's API ecosystem. The reconstruction began by revitalizing core endpoints—including canteen services (sas), RSS-to-JSON translation tools (Rss2Json), institutional data (ArcGIS), real-time parking availability (parques), and academic services (sac). By employing current programming languages and libraries, we addressed the fundamental bugs and inefficiencies of the previous system. To safeguard these assets against external threats, the new APIs were integrated into a robust WSO2 management layer, unified under the university's proprietary identity provider (IdpUa). The result is a secure, high-performance platform complemented by a dedicated documentation portal and an iterative testing framework, ensuring that the university's digital tools are as reliable and accessible as the services they represent.

Contents

Abstract **1**

1 Introduction **6**
1.1 Context and Motivation 6
1.2 Problem Statement 6
1.3 Goals and Objectives 7
1.4 Document Structure(mudar) 7

2 Related Work, Tools and Technologies **8**
2.1 Related Work 8
2.1.1 Commercial API Management Platforms 9
2.1.2 Comparable Institutional Platforms 9
2.2 Tools and Technologies 10
2.2.1 Python and FastAPI 10
2.2.2 WSO2 API Manager 10
2.2.3 OAuth2 and `idp.ua.pt`(agora é keycloack não?) 10
2.2.4 Docusaurus and Stoplight Elements 10
2.2.5 Memcached 11
2.2.6 Docker and Kubernetes(usamos kubernetes?) 11
2.2.7 Grafana and Loki 11
2.2.8 SQL 11

3 Requirements Elicitation and Architecture **12**
3.1 Requirements Elicitation 12
3.1.1 Functional Requirements 12
3.1.2 Non-Functional Requirements 13
3.2 System Architecture(mudar todos os pontos quase) 13
3.2.1 User Interface — Academic Playground & Innovation 14
3.2.2 WSO2 API Gateway Layer 15
3.2.3 FastAPI Backend - ServicesUAPT 15
3.2.4 Mock API Access Points 16

3.2.5 Data Sources (Backend) 16

4 Expected Results(mudar?) 18

4.1 Projected Schedule 19

List of Figures

3.1 System Architecture Overview 13

4.1 Projected Schedule 19

List of Tables

Chapter 1

Introduction

1.1 Context and Motivation

The University of Aveiro has long relied on a centralised web services platform — hosted at `api.web.ua.pt` — to expose institutional data to students, faculty, and third-party developers. Through this platform, the university made available a range of services that became embedded in the daily life of its academic community: real-time canteen menus, academic services queue tickets, campus building information, and parking availability, among others.

Despite its relevance, the platform was built over a decade ago on a PHP stack that has since reached end-of-life. Over time, the absence of security updates, the accumulation of unresolved bugs, and the growing incompatibility with modern development practices rendered the system increasingly difficult to maintain. These factors ultimately led to the decommissioning of the legacy platform, leaving a gap in the university's digital infrastructure that directly affected students and developers who depended on it.

1.2 Problem Statement

The core issues that led to the platform's decommissioning can be grouped into three categories.

The first is **security**. The legacy system ran on an outdated version of PHP with known CVEs that had gone unpatched, exposing the platform to exploitation. With no authentication or authorisation layer in place, any consumer could call any endpoint without restriction, making it impossible to protect sensitive data or audit access.

The second is **reliability and performance**. The architecture was not designed to handle concurrent load gracefully. Under peak usage — such as at the start of the academic year or during lunch hours — the server would become unresponsive,

degrading the experience for all users simultaneously.

The third is **maintainability**. The codebase lacked modularity and documentation, making it difficult to onboard new developers or extend the platform with additional services. Adding a new API required modifying a tightly coupled codebase with no clear separation of concerns.

1.3 Goals and Objectives

This project aims to deliver a complete, production-ready replacement for the legacy platform. The main objectives are:

- Migrate all existing services to a modern Python stack using FastAPI, preserving the original endpoints to ensure backward compatibility with existing consumers;
- Introduce a WSO2 API Gateway layer to centralise authentication, authorisation, monitoring, and rate limiting across all services;
- Develop a documentation and interactive testing portal — the *Academic Playground & Innovation* — that makes it straightforward for students and developers to discover, understand, and integrate the available APIs;
- Design the architecture to be modular and extensible, so that new services can be added with minimal effort and no disruption to existing ones.
- **FAZER A NOVA API MOCK?**

1.4 Document Structure(mudar)

This report is organised into four chapters. Chapter 2 surveys existing solutions and API gateway platforms, and presents the tools and technologies selected for the project, justifying each choice. Chapter 3 covers the requirements elicitation process — both functional and non-functional — and provides a detailed description of the system architecture and its constituent layers. Chapter 4 outlines the expected outcomes of the project, the planned deliverables, the evaluation criteria, and the development schedule.

Chapter 2

Related Work, Tools and Technologies

2.1 Related Work

Two previous projects developed at the University of Aveiro are particularly relevant to this work: the *OPEN DATA UA* master thesis and the final report of the *Academic Playground & Innovation (API)* project. Together, they provide both the conceptual and practical foundations on which this project is built.

The *OPEN DATA UA* thesis surveys existing institutional information sources and web services at UA and proposes a service-oriented architecture based on REST and SOAP for exposing institutional data (e.g., canteen menus, academic information) in a standardized way. It also implements a web portal (APIn) that aggregates services from UA, PT Inovação and SAPO, and introduces an OAuth-based authorization server to protect access to private or sensitive web services. This work is mainly focused on identifying data sources, creating individual web services, and demonstrating how open data and secure access control can be applied within the university context.

The *Academic Playground & Innovation* final report documents the concrete implementation of that vision, namely the api.web.ua.pt portal and the supporting services running on services.web.ua.pt and identity.ua.pt. It describes in detail the API portal, the catalogue of services (UA, PT SAPO, PT Inovação), the documentation and wiki mechanisms, the statistics and administration interfaces, as well as the deployment of an OAuth 1.0a server integrated with the UA Identity Provider using SimpleSAMLphp. This report provides an operational view of how service publication, authentication, authorization and consumption were handled in practice in the first generation of the institutional API platform.

This project can be seen as a next step in this evolution. While *OPEN DATA UA* and the API portal focused on exposing individual services and providing a web-based aggregation and documentation layer, they are tightly coupled to specific technologies and infrastructures that have since become obsolete. In contrast, this project aims

at designing a modern institutional API Gateway and governance model, aligned with current API management practices. Building on the lessons learned from these previous projects, the goal is to provide a more sustainable, secure and governable integration layer for present and future UA systems.

2.1.1 Commercial API Management Platforms

Several commercial and open-source API management platforms address problems similar to those this project aims to solve. **Kong Gateway** is a widely adopted open-source API gateway, offering plugins for authentication, rate limiting, logging, and traffic routing. It is highly performant and extensible, but requires significant infrastructure investment and operational expertise to deploy and maintain at scale. Given that one of the core goals of this project is long-term sustainability and ease of handover to future development teams, introducing a platform with such operational complexity would be counterproductive.

AWS API Gateway and **Azure API Management** are fully managed cloud offerings that eliminate the operational burden of running gateway infrastructure. Both provide built-in support for OpenAPI specifications, OAuth2 integration, and developer portals. However, as hosted proprietary services, they introduce vendor lock-in and are unsuitable for an institutional deployment that must remain under the university's control and on-premises.

RapidAPI represents a different model, functioning primarily as a marketplace and developer portal for discovering and testing third-party APIs. Its interactive testing interface — which allows developers to issue live requests directly from the documentation page — was a direct source of inspiration for the *Academic Playground & Innovation* portal developed in this project.

2.1.2 Comparable Institutional Platforms

Beyond commercial offerings, several universities and public institutions have developed API platforms to expose institutional data to their communities. MIT's *Atlas* and Stanford's developer portals follow a similar model: centralised access to institutional data through authenticated APIs, with developer documentation and sandbox environments.

The common thread across these platforms is the separation between the *gateway layer* — responsible for security, routing, and governance — and the *backend services* that implement the actual business logic. This separation is a key architectural principle adopted in this project, and one that was notably absent from the legacy system, where security concerns, business logic, and data access were all handled within the same tightly coupled PHP codebase with no clear boundaries between layers.

2.2 Tools and Technologies

2.2.1 Python and FastAPI

Python was selected as the primary implementation language for the backend services. Its extensive ecosystem, readability, and widespread adoption in both academia and industry make it a natural fit for a platform that must be maintained by successive generations of students. FastAPI is a modern Python web framework designed for building APIs with high performance and minimal boilerplate. It leverages Python type annotations to generate OpenAPI-compliant documentation automatically at runtime, eliminating the need to maintain separate API specification files. This tight integration between code and documentation is particularly valuable in a context where the OpenAPI specification is consumed directly by both the WSO2 gateway and the developer portal.

2.2.2 WSO2 API Manager

WSO2 API Manager is an open-source, on-premises API management platform that provides a complete solution for publishing, securing, monitoring, and governing APIs. It was selected for this project because it supports the full OAuth2 authorization code flow natively, integrates with external identity providers via OpenID Connect — enabling seamless federation with UA's `idp.ua.pt` — and exposes a standards-based management interface that can ingest OpenAPI specifications directly. Its open-source nature and on-premises deployment model ensure that the university retains full control over the platform without dependency on third-party cloud providers.

2.2.3 OAuth2 and `idp.ua.pt`(agora é keycloack não?)

Authentication and authorisation are handled through the OAuth2 protocol, delegated to the University of Aveiro's institutional identity provider (`idp.ua.pt`). This approach ensures that access to private APIs is gated behind the same credentials used across all university systems, without requiring users to manage separate accounts. The WSO2 gateway acts as the OAuth2 resource server, validating tokens issued by `idp.ua.pt` before forwarding requests to the backend.

2.2.4 Docusaurus and Stoplight Elements

The developer portal is built with **Docusaurus**, a React-based static site generator maintained by Meta. It was chosen for its native support for Markdown and MDX content, built-in versioning, and low maintenance overhead — documentation updates require only editing text files, with no application code changes. Interactive API testing

is provided by **Stoplight Elements**, an open-source React component that renders a full API explorer from an OpenAPI specification. All test requests issued through the portal are routed through the WSO2 gateway, ensuring that the developer experience accurately reflects production behaviour.

2.2.5 Memcached

Memcached is a high-performance, in-memory key-value store used in this project to cache upstream API responses. Given that certain data sources — such as the UA WSO2 canteen menu endpoint — are slow to respond and return data that changes at most once per day, caching responses eliminates redundant upstream calls and significantly reduces response times for end consumers. Memcached was preferred over alternatives such as Redis for its simplicity and minimal resource footprint, which is particularly relevant given the constrained hardware available for deployment.

2.2.6 Docker and Kubernetes(usamos kubernetes?)

All backend services are containerised using **Docker**, ensuring environment consistency across development, testing, and production. Orchestration is handled by **Kubernetes**, using the cluster infrastructure provided by the University of Aveiro. Kubernetes enables declarative deployment, automatic restarts on failure, horizontal scaling, and rolling updates with no downtime — addressing the reliability and scalability requirements that the legacy platform was unable to meet.

2.2.7 Grafana and Loki

For monitoring and logging, the platform integrates with **Grafana** and **Loki**.

2.2.8 SQL

Chapter 3

Requirements Elicitation and Architecture

3.1 Requirements Elicitation

The requirements for this project is a very important part of the development process, as they will guide the design and implementation of the system. To gather these requirements, we talked to STIC (the most important stackeholder in this project) and our supervisor, Diogo Gomes.

3.1.1 Functional Requirements

- **FR1 – Service Centralization and Exposure:** A WSO2 layer must exist that acts as a centralized API Gateway to expose the University of Aveiro's services uniformly.
- **FR2 – Identity and Permissions:** The system must implement authentication and authorization mechanisms, ensuring access control per API according to each user's profile.
- **FR3 – Control and Monitoring:** The platform must allow detailed control and monitoring of all service accesses, including logging and traceability of calls made.
- **FR4 – Documentation Availability:** Documentation must be available for the use of each API, including description of endpoints, parameters, authentication, and usage examples.
- **FR5 – Continuity and Compatibility:** The platform must maintain the same endpoints, ensuring compatibility with already integrated systems and avoiding service interruptions.

3.1.2 Non-Functional Requirements

- **NFR1 – Security:** The system must comply with security best practices, mitigating vulnerabilities identified in the previous solution and protecting data and access.
- **NFR2 – Performance:** API requests must be processed, under normal operating conditions, in less than 3 seconds.
- **NFR3 – Technical Scalability:** The API Gateway must support increased load and growth in the number of consumers without significant service degradation.
- **NFR4 – Functional Scalability:** The introduction and integration of new APIs must be simple, quick, and have minimal impact on existing services.
- **NFR5 – Maintainability:** The system must adopt software development best practices and keep code readable, modular, and well-documented to facilitate corrective, evolutionary, and preventive maintenance.
- **NFR6 – Interface Usability:** The documentation portal must present a simple, clear, and intuitive interface, promoting the adoption and use of available services.

3.2 System Architecture(mudar todos os pontos quase)

The platform is organised into four distinct layers, as illustrated in Figure 3.1: the *User Interface*, the *WSO2 API Gateway*, the *FastAPI backend*, and the *Backend data sources*.

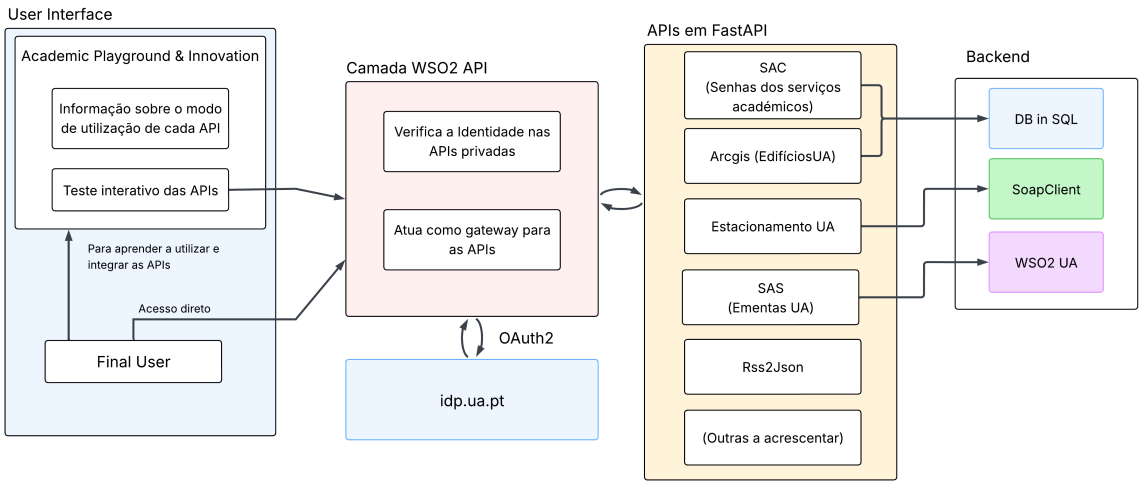

Figure 3.1: System Architecture Overview

3.2.1 User Interface — Academic Playground & Innovation

The user-facing layer is the *Academic Playground & Innovation* portal. It serves two purposes: providing documentation on how to use each API, and allowing interactive testing of all available endpoints directly in the browser. The goal here is to make it easy for people to learn how to integrate the APIs available in their own projetcs, as after they are familiar with the methods there is no real use for this layer.

The portal was developed using **Docusaurus**, a static site generator built on React and maintained by Meta. This choice is justified by its native support for Markdown and MDX, built-in documentation versioning, and ease of maintenance — updating content requires only editing text files, with no need to touch application code, this also facilitates escalating the number of APIs, as it's easy to make a complete documentation about all it's methods. The *Academic Playground & Innovation* portal provides interactive testing of all available endpoints directly in the browser, following the same approach used by platforms such as RapidAPI (wich heavily inspired this front-end). This is achieved by embedding **Stoplight Elements** into the Docusaurus site, which renders a full API explorer from the OpenAPI specification generated automatically by FastAPI.

Rather than communicating with the FastAPI backend directly, all test requests issued from the portal are routed through the **WSO2 API Gateway**, the same path taken by any external consumer. This is accomplished by setting the WSO2 public endpoint as the target server in the OpenAPI specification:

Listing 3.1: OpenAPI server configuration targeting WSO2

```
{
  "servers": [
    { "url": "https://api.ua.pt/v1" }
  ]
}
```

With this configuration, the portal's test interface reflects the real behaviour of the platform in production — including authentication, rate limiting, and request routing — rather than bypassing the gateway layer. For public APIs, such as the canteen menus, no credentials are required and requests can be issued immediately. For private APIs, Stoplight Elements supports OAuth2 flows natively, allowing users to authenticate with their institutional credentials via `idp.ua.pt` and have the resulting token attached automatically to all subsequent test requests.

3.2.2 WSO2 API Gateway Layer

All requests from the portal pass through a WSO2 API Manager instance, which acts as the central API Gateway for the platform. For the test requests explained in the previous layer to function correctly from a browser context, the WSO2 instance must be configured to include the appropriate `Access-Control-Allow-Origin` headers in its responses, permitting cross-origin requests originating from the portal's domain. This layer is responsible for:

- **Authentication and authorisation** — identity verification for private APIs is delegated to `idp.ua.pt` via OAuth2, this ensures institutional credentials are being requested correctly from UA's official identity provider;
- **Request routing** — the gateway forwards validated requests to the appropriate ServicesUAPT service;
- **Monitoring and rate limiting** — all inbound traffic is logged and can be controlled centrally without changes to the backend.

Public APIs (such as the canteen menus) are accessible without authentication, while APIs that expose sensitive data require a valid OAuth2 token issued by the university's identity provider.

3.2.3 FastAPI Backend - ServicesUAPT

The core logic of each service is implemented in Python using the FastAPI framework. This layer is completely replacing the old ServicesUAPT wich was very outdated. Each service is isolated in its own router module, making it straightforward to add, remove, or update individual APIs without affecting the rest of the platform. The services currently implemented or planned are:

- **SAC** — queue ticket numbers for the Academic Services office;
- **ArcGIS** — data about university buildings and campus units;
- **Estacionamento UA** — real-time parking availability;
- **SAS** — canteen menus, consumed from the UA WSO2 API and cached via Memcached;
- **Rss2Json** — generic RSS feed to JSON converter;
- (Additional services to be integrated in future iterations).

FastAPI was chosen because it generates OpenAPI-compliant documentation automatically, which integrates directly with the WSO2 gateway and removes the need to maintain separate API specifications.

3.2.4 Mock API Access Points

The API UA platform is not merely a system that centralizes the various services offered by the University of Aveiro. Rather, it is designed to facilitate easy integration of new APIs that may be developed in the future. To demonstrate this extensibility, we created a mock API to simulate the integration of new web services into the platform.

Our supervisor recommended developing a mock API for the University of Aveiro's access points, containing useful information such as their geographical locations, maximum connection capacity, and other data relevant to the academic community.

To store the mock API data we created a MySQL database hosted on the IETTA infrastructure. This decision was driven by the fact that, concurrently with this project, we attended a relational databases course that provided access to the IETTA database for class exercises and the course project. Given that existing access and the operational convenience it provided, we obtained permission from the course coordinator to use the IETTA database for persisting the data for our mock API.

The database schema contains two primary tables: one for access points and another for the departments where each access point is located. Each table includes several attributes, which are represented in the entity–relationship diagram (ERD) shown in Figure (...). The ERD was designed to organise and document the database structure.

Flask, environment python, DER -Carlos

The following architecture was designed:

–> Arquitetura(foto) - Pedro

(EM DESENVOLVIMENTO)

3.2.5 Data Sources (Backend)

Depending on the service, the FastAPI layer consumes data from one of this sources:

- **CSV File** — used by the SAC service for persistent storage of queue ticket data, written and read directly from a local CSV file;
- **SOAP Client** — used by the SAC service to interface with the legacy university systems that expose SOAP/XML web services;
- **WSO2 UA** — the university's own API Gateway, consumed by services such as SAS to retrieve data like canteen menus and parking availability.
- **SQL** — ola ola

The platform is containerised using Docker and deployed on the University of Aveiro's Kubernetes cluster, ensuring availability, scalability, and ease of rolling updates.

Chapter 4

Expected Results(mudar?)

At the end of this project the expected result is to have a fully operational platform that operates in accordance with the objectives that were proposed. The system will be completely secured, operating with the university's identity provider (IdpUA), to prevent any unwanted access to the private APIs implemented. The platform should also be intuitive and easy to use by students, professors and external developers alike that aim to integrate any of the available APIs into their own projects. To support this, a comprehensive documentation website will be created, covering each endpoint and method available, alongside an interactive testing mechanism that allows users to try all those methods directly in the browser. Due to the modernization and different implementations, such as the usage of Memcached caching and other technologies, responses are expected to be processed in under 3 seconds under normal operating conditions. Finally, all the work done will remain compatible with systems already integrated with the previous platform, as the endpoints will stay the same, avoiding service disruption and ensuring continuity for existing consumers.

4.1 Projected Schedule

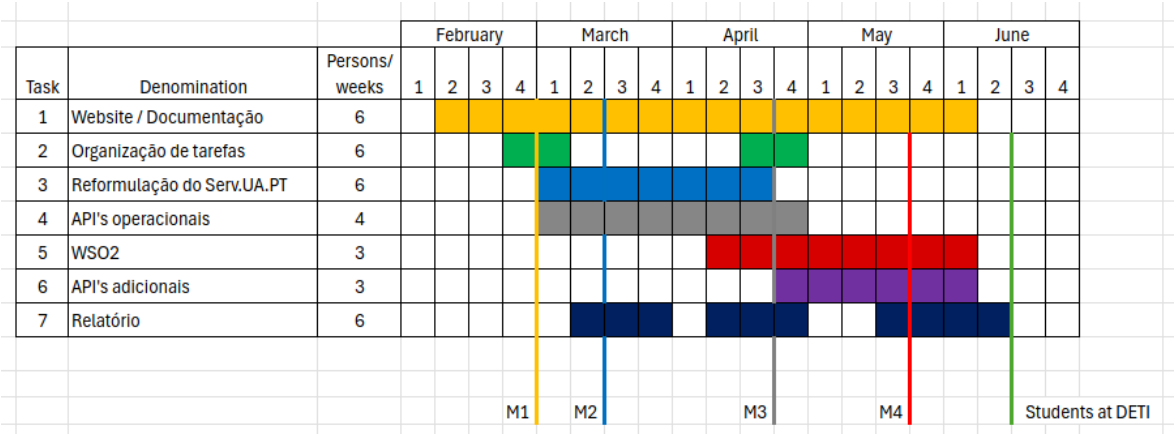

Figure 4.1: Projected Schedule

The planned work is organised into seven main tasks, distributed between February and June. Website and documentation activities span almost the entire period, providing continuous support to the remaining tasks. Initial weeks focus on task organisation and the refactoring of existing services, followed by the implementation and stabilisation of the core institutional APIs. In the final phase, additional APIs are integrated and the project report is completed, ensuring that the technical work is accompanied by up-to-date documentation and validation milestones.

Bibliography